Studium
Enciclopedia de Programación e Ingeniería del Software

Volumen V: Monografías de Python, JavaScript y TypeScript

Parte V (Bloques A–H): De cero a máquinas virtuales de bytecode, asincronía, Event Loop V8 y sistemas de tipos estructurales

Borrador
Edición 0.1.0 — No publicado
20 de octubre de 2026

Prefacio

Este libro presenta el tema con el lenguaje del curso. Cada capítulo abre con una entrada breve, sigue con la explicación y cierra con un problema resuelto y una autoficha.

Cómo usar este libro

Lee el capítulo seguido. El consejo, la definición, el problema resuelto y la autoficha van en recuadros. La explicación es el cuerpo del texto.
Un capítulo lleva una entrada en cursiva, la explicación en el cuerpo, como mucho un consejo, definiciones solo al introducir un término, un problema resuelto y una autoficha.

Índice

Prefacio II

Cómo usar este libro III

I Monografía Autónoma de Python (Python 3.12 – 3.14) 1

1. Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante 2
1.1. Bloques A y B: Historia, máquina virtual CPython, GIL / Free-Threading y toolchain moderna (uv, ruff, pyright) . . . 2
1.2. Bloque C: Modelo de nombres y objetos, tipos nativos, colecciones, comprensiones y control de flujo (match/case) . . . 3

2. Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers 5
2.1. Bloque D: POO en Python, Data Model (___dunder___), @dataclass, ___slots___ y sistema de tipos (PEP 484 / PEP 695) . . . 5
2.2. Bloque E: Clausuras, decoradores, iteradores, generadores (yield) y gestores de contexto (with) . . . 6

3. Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos 8
3.1. Bloque F: Asincronía con asyncio.TaskGroup, multiprocesamiento, metaclases y compilación AST . . . 8
3.2. Bloques G y H: Ecosistema industrial (FastAPI, Pydantic, SQLAlchemy, Pytest) y los 4 proyectos (python-bytecode-vm) . . . 9

II Monografías Autónomas de JavaScript (ES2024+) y TypeScript (5.x) 11

4. Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM 12
4.1. Bloques A–C: Historia de ECMAScript, motores JIT (V8), tipos, coerción vs ===, let/const y funciones . . . 12
4.2. Bloque D: Clausuras, delegación por prototipos, clases, desestructuración y módulos ES (ESM vs CommonJS) . . . 12

5. Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos 15
5.1. Bloques E y F: Event Loop, cola de microtareas vs macrotareas, Promises, async/await, Streams y Workers . . . 15
5.2. Bloques G y H: Ecosistema (Node.js, Deno, Bun, npm/pnpm, Vitest, ESLint) y los 4 proyectos en JavaScript . . . 16

6. Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas 18
6.1. Bloques A–C: Filosofía de TypeScript, borrado de tipos, tsconfig.json estricto y primitivos (unknown, never) . . . 18
6.2. Bloque D: Tipado estructural, interfaces vs type aliases, estrechamiento (narrowing) y uniones discriminadas . . . 18

7. Monografía de TypeScript (II − Bloques E−H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos **21**
7.1. Bloques E y F: Generics, keyof, Mapped Types, Conditional Types, infer, Template Literals, satisfies y Branded Types . . . 21
7.2. Bloques G y H: Ecosistema (Zod, tRPC, Vite, tsx, Vitest) y los 4 proyectos (incluyendo ts-reactive-http) . . . 22

8. Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript **24**
8.1. Comparativa profunda de ejecución, memoria, concurrencia y sistemas de tipos (Python vs JS vs TS) . . . 24
8.2. Arquitectura políglota: contratos OpenAPI/JSON Schema entre Python y TypeScript y evaluación global . . . 24

A. Notación **27**

B. Hoja de fórmulas **28**

C. Soluciones **29**
C.1. Monografía de Python (I − Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante . . . 29
C.2. Monografía de Python (II − Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers . . . 29
C.3. Monografía de Python (III − Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos . . . 30
C.4. Monografía de JavaScript (I − Bloques A−D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM . . . 31
C.5. Monografía de JavaScript (II − Bloques E−H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos . . . 31
C.6. Monografía de TypeScript (I − Bloques A−D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas . . . 32
C.7. Monografía de TypeScript (II − Bloques E−H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos . . . 33
C.8. Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript . . . 33

D. Auditoría de fuentes **35**

E. Plan de estudio **37**

Parte I

Monografía Autónoma de Python (Python 3.12 – 3.14)

Capítulo 1

Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante

*Este capítulo inaugura la monografía autónoma de **Python** (Bloques A, B y C). Python es simultáneamente el lenguaje hegemónico en ciencia de datos, inteligencia artificial y automatización, y un lenguaje de ingeniería de software de primer orden para backends web y herramientas de infraestructura. El lector aprenderá su modelo exacto de ejecución bajo el capó de CPython.*

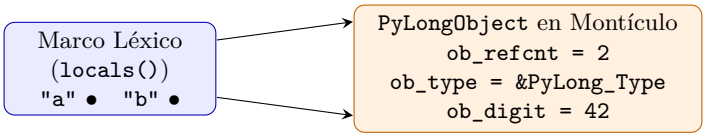


Figura 1. Diferencia semántica fundamental en Python: las variables no son cajas con tipo fijo, sino etiquetas en un diccionario de espacio de nombres que apuntan a estructuras PyObject en el montículo con cabecera $\mathrm{ob}_r efcnt$ y $\mathrm{ob}_t ype$.

Consejo

Utiliza siempre `is None` e `is not None` para comprobar identidad frente al singleton `None`, y reserva `==` para comparar igualdad de valor (`__eq__`).

Definición

Vinculación de nombres en Python (Name Binding): asociación en una tabla de símbolos (`locals()`, `globals()`) entre un identificador y la dirección de un `PyObject` en el montículo; el tipo pertenece exclusivamente al objeto referenciado y nunca a la variable léxica.

Autoficha

¿Por qué una tupla `(1, [2, 3])` lanza `TypeError: unhashable type: 'list'` al intentar usarla como clave de un `dict` a pesar de que el contenedor exterior es un `tuple` inmutable?

1.1 Bloques A y B: Historia, máquina virtual CPython, GIL / Free-Threading y toolchain moderna (uv, ruff, pyright)

Creado por Guido van Rossum en el CWI de Ámsterdam y publicado en 1991, Python ejecuta el código fuente compilándolo primero a un árbol de sintaxis abstracta (AST) y seguidamente a código de bytes (*bytecode*, inspeccionable con el módulo estándar `dis`) interpretado por una máquina virtual de pila en CPython. Cada valor en Python −incluidos los enteros y las propias clases− es una estructura `PyObject` en el montículo que almacena como mínimo su contador de referencias (`ob_refcnt`, 8 bytes en 64 bits) y un puntero a su descriptor de tipo (`ob_type`, 8 bytes), como ilustra la Figura \ref{fig:fig-cpython-object-layout}. La gestión de memoria combina liberación inmediata cuando `ob_refcnt` cae a cero con un recolector generacional detector de ciclos. Históricamente, CPython protegió el estado del intérprete con el cerrojo global **GIL** (*Global Interpreter Lock*), pero a partir de Python 3.13/3.14 (PEP 703) incorpora soporte para compilaciones *free-threaded* sin GIL basadas en conteo de referencias sesgado (*biased reference counting*). Todo proyecto moderno de Python se configura mediante un único fichero declarativo `pyproject.toml` (PEP 621), gestionado con `uv` o `pip` sobre `venv`, formateado y auditado con `ruff`, verificado estáticamente con `pyright` o `mypy` y probado con `pytest`.

1.2 Bloque C: Modelo de nombres y objetos, tipos nativos, colecciones, comprensiones y control de flujo (match/case)

En Python, la sentencia `a = [1, 2, 3]; b = a` **no** copia la lista: vincula ambos nombres al mismo objeto `PyListObject` incrementando su `ob_refcnt` a 2. La distinción entre tipos inmutables (`int` de precisión arbitraria, `float` IEEE 754, `str` Unicode, `bytes`, `tuple`, `frozenset`) y mutables (`list`, `dict`, `set`, `bytearray`) determina qué objetos son *hashables* y pueden actuar como claves de un diccionario. Desde Python 3.6/3.7, `dict` se implementa como una tabla hash compacta que preserva por especificación el orden de inserción separando un array denso de índices hash de la tabla contigua de entradas `(hash, key_ptr, val_ptr)`. Asimismo, Python 3.10+ incorpora coincidencia estructural de patrones (`match/case`, PEP 634):

```
def evaluate_ast_node(node: tuple) -> int:
    match node:
        case ("lit", int(v)):
            return v
        case ("add", left, right):
            return evaluate_ast_node(left) + evaluate_ast_node(right)
        case ("mul", left, right):
            return evaluate_ast_node(left) * evaluate_ast_node(right)
        case _:
            raise ValueError(f"Nodo AST desconocido: {node!r}")
```

Un escollo clásico que todo programador debe evitar desde el primer día es emplear argumentos por defecto mutables en definiciones de función (`def append_item(x, target=[]):`), ya que la expresión por defecto se evalúa una única vez al ejecutar la sentencia `def`, compartiendo la misma lista entre todas las llamadas; el patrón correcto es `def append_item(x, target: list | None = None):`.

Problema resuelto

Enunciado En CPython de 64 bits, cada entrada de un diccionario almacena tres palabras de 8 B (`me_hash`, `me_key`, `me_value` = 24 B). Para una tabla con $K = 128$ ranuras en el índice (1 B por ranura en `dk_indices`) y capacidad para $N = 85$ entradas en `dk_entries`, calcula cuántos bytes ahorra el diseño de diccionario compacto ($128 \cdot 1 + 85 \cdot 24$) frente al diseño disperso antiguo ($128 \cdot 24$).
Datos

- $K = 128$ ranuras — Tamaño del array de índices hash (1 B por índice)
- $N = 85$ entradas — Capacidad densa de entradas (2/3 de K)
- $S_{entry} = 24$ B — Tamaño de cada entrada (hash, key, value)

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de memoria del diccionario compacto de CPython (Raymond Hettinger)

$$\Delta M = 128 \cdot 24 - (128 \cdot 1 + 85 \cdot 24)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el tamaño compacto (2168 B) al tamaño antiguo (3072 B)

$$\Delta M = 128 \cdot 24 - (128 \cdot 1 + 85 \cdot 24)$$

$$((128 \cdot 24) - ((128 \cdot 1) + (85 \cdot 24)))$$

Resultado

$$Resultado = 904$$

Interpretación El diccionario compacto ahorra 904 bytes (casi un 30 % menos de memoria) y acelera la iteración al recorrer un array contiguo.
Comprobación de los pasos

- Paso 1: 904 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0003] (Básico, Numérico) En CPython de 64 bits, la cabecera base `PyObject` contiene `ob_refcnt` (8 B) y `ob_type` (8 B). ¿Cuántos bytes mide esta cabecera mínima?
2 [PRB-0004] (Básico, Numérico) Una lista `PyListObject` mantiene un array de punteros `PyObject*` (8 B cada uno). Si su capacidad reservada es de 64 elementos, ¿cuántos bytes ocupa ese array de punteros?
3 [PRB-0005] (Intermedio, Desarrollo por pasos) ¿Cuál es la suma de los cuadrados impares evaluada por `sum(x * x for x in range(1, 10, 2))` ($1^2 + 3^2 + 5^2 + 7^2 + 9^2$)?
4 [PRB-0006] (Intermedio, Diseño) En el proyecto `projects/python-bytecode-vm`, la función compila y evalúa `(x + 4) * (y - 2) + (10 if x < y else 20)` para $x = 6, y = 9$. ¿Cuál es el resultado entero devuelto?

Capítulo 2

Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers

*Este capítulo cubre los Bloques D y E de Python. La verdadera fluidez en Python consiste en comprender que la sintaxis del lenguaje (`len(x)`, `x[i]`, `for v in x`, `with ctx:`, `a + b`) se traduce directamente en invocaciones a métodos especiales del **Modelo de Datos** (`__len__`, `__getitem__`, `__iter__`, `__enter__`/`__exit__`, `__add__`).*

Consejo

Usa siempre `@dataclass(slots=True)` en clases de datos de alto volumen: reduce el consumo de memoria entre un 40 % y un 60 % y acelera el acceso a atributos al sustituir búsquedas hash en `__dict__` por desplazamientos directos en C.

Definición

Protocolo Descriptor en Python: mecanismo fundamental del modelo de objetos según el cual cualquier atributo de clase que defina `__get__`, `__set__` o `__delete__` intercepta el acceso a atributos en las instancias, sirviendo de base para métodos ligados, `@property`, `@classmethod`, `@staticmethod` y `__slots__`.

Autoficha

¿Qué diferencia existe en el protocolo descriptor entre un *data descriptor* (define `__get__` y `__set__`) y un *non-data descriptor* (solo define `__get__`) frente a una entrada con el mismo nombre en `instance.__dict__`?

2.1 Bloque D: POO en Python, Data Model (___dunder___), @dataclass, ___slots___ y sistema de tipos (PEP 484 / PEP 695)

Por defecto, cada instancia de una clase definida por el usuario almacena sus atributos en un diccionario dinámico `__dict__`, lo que aporta flexibilidad pero añade más de 100 B de sobrecoste por objeto. Cuando se instancian millones de objetos de dominio o nodos del AST (como en nuestro proyecto `projects/python-bytecode-vm`), declarar `@dataclass(frozen=True, slots=True)` genera automáticamente `__init__`, `__repr__`, `__eq__`, `__hash__` y reemplaza `__dict__` por un descriptor compacto de ranuras fijas (`__slots__`). Asimismo, Python 3.12 (PEP 695) simplifica la sintaxis de clases y funciones genéricas y de `typing.Protocol` (subtipado estructural o *static duck typing*):

```
from dataclasses import dataclass
from typing import Protocol

class Measurable(Protocol):
    def byte_size(self) -> int: ...

@dataclass(frozen=True, slots=True)
class TelemetryFrame:
    sensor_id: int
```

```
    payload: bytes
    def byte_size(self) -> int:
        return 8 + len(self.payload)

def total_footprint[T: Measurable](items: list[T]) -> int:
    return sum(item.byte_size() for item in items)
```

Observa que `TelemetryFrame` satisface `Measurable` estructuralmente para `pyright`/`mypy` sin necesidad de heredar nominalmente de `Measurable`.

2.2 Bloque E: Clausuras, decoradores, iteradores, generadores (yield) y gestores de contexto (with)

Un **generador** es cualquier función cuyo cuerpo contiene la palabra clave `yield`. Al llamarla, no ejecuta su cuerpo inmediatamente, sino que devuelve un objeto iterador que suspende y reanuda su marco de ejecución (`PyGenObject`) en cada llamada a `next()`, manteniendo un consumo de memoria $O(1)$ independientemente de si la secuencia subyacente tiene miles de millones de registros. Por su parte, los **gestores de contexto** implementan el protocolo `__enter__` / `__exit__(exc_type, exc_val, exc_tb)` (o se escriben de forma concisa con `@contextlib.contextmanager` y un único `yield` dentro de un bloque `try ... finally:`), garantizando la liberación determinista de ficheros, conexiones de red, cerrojos `threading.Lock` y transacciones de base de datos.

Problema resuelto

Enunciado En una aplicación de procesamiento de grafos en CPython de 64 bits, una clase `Edge` con $k = 3$ atributos ocupa $16 + 3 \times 8 = 40$ B cuando se declara con `__slots__`, mientras que sin `__slots__` cada instancia aloja además un `__dict__` que añade 112 B por objeto (152 B en total). Calcula cuántos megabytes exactos (dividiendo entre 10^6) se ahorran al instanciar $N = 1000000$ de aristas con `__slots__`.

Datos

- $N = 1000000$ — Un millón de instancias de Edge
- $S_{condict} = 152\,\text{B}$ — Tamaño por instancia con ___dict___
- $S_{conslots} = 40\,\text{B}$ — Tamaño por instancia con ___slots___

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de RAM con ___slots___ en millones de instancias en CPython

$$\Delta M_{\text{MB}} = \frac{10^6 \cdot (152 - 40)}{10^6}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos el millón de instancias por los 112 B ahorrados y dividimos entre 10^6

$$\Delta M_{\text{MB}} = \frac{10^6 \cdot (152 - 40)}{10^6}$$

$$\frac{(1000000 \cdot (152 - 40))}{1000000}$$

Resultado

$$\textit{Resultado} = 112$$

Interpretación El uso de ___slots___ ahorra exactamente 112 MB de memoria RAM.

Comprobación de los pasos

- Paso 1: 112 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0009] (Básico, Numérico) En CPython de 64 bits, ¿cuántos bytes ocupa una instancia con `__slots__` de 4 atributos (16 B de cabecera más 4×8 B)?
2 [PRB-0010] (Básico, Numérico) Un generador perezoso produce bloques de 4096 bytes uno a uno para procesar un fichero de 10 GiB. ¿Cuántos bloques mantiene simultáneamente en RAM?
3 [PRB-0011] (Intermedio, Desarrollo por pasos) En una jerarquía con herencia múltiple `class D(B, C)` donde `B(A)` y `C(A)` heredan de `A(object)`, ¿cuántas clases aparecen en la tupla de orden de resolución de métodos `D.__mro__` (`D, B, C, A, object`)?
4 [PRB-0012] (Intermedio, Diseño) Un decorador `@lru_cache(maxsize=256)` recibe 1000 consultas sobre un conjunto de solo 200 claves distintas. ¿Cuántas veces se ejecuta realmente la función subyacente?

Capítulo 3

Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos

Este capítulo completa la monografía de Python abordando el Nivel Experto (Bloque F), el Ecosistema (Bloque G) y los Proyectos (Bloque H), analizando en detalle el compilador AST y la máquina virtual de código de bytes `projects/python-bytecode-vm`.

Consejo

Nunca llames a funciones bloqueantes síncronas (`time.sleep`, `requests.get`, lectura síncrona de disco) dentro de una función `async def`: bloquearás todo el bucle de eventos de `asyncio` congelando el resto de tareas concurrentes; utiliza `await asyncio.sleep(...)`, `httpx.AsyncClient` o `await asyncio.to_thread(...)`.

Definición

Concurrencia estructurada (`asyncio.TaskGroup`): abstracción de control de flujo concurrente que garantiza que el tiempo de vida de todas las tareas asíncronas lanzadas dentro de un bloque léxico `async with` está acotado por dicho bloque, sin dejar tareas huérfanas ejecutándose en segundo plano tras salir del ámbito.

Autoficha

¿Por qué en `projects/python-bytecode-vm` un salto condicional `JMP_IF_FALSE` necesita parchar (*backpatch*) su índice de destino después de haber emitido las instrucciones de la rama verdadera?

3.1 Bloque F: Asincronía con asyncio.TaskGroup, multiprocesamiento, metaclases y compilación AST

En Python, la elección del modelo de concurrencia depende de la naturaleza del cuello de botella: para operaciones limitadas por E/S de red o disco (*I/O-bound*) con miles de conexiones, `asyncio` ejecuta corrutinas cooperativas sobre un único hilo respaldado por `epoll`/`kqueue`/`iocp`. Python 3.11+ introdujo **concurrencia estructurada** mediante `async with asyncio.TaskGroup() as tg:`: si cualquiera de las tareas hijas lanza una excepción no capturada, el `TaskGroup` cancela automáticamente todas las tareas hermanas y propaga un `ExceptionGroup` manejable con `except*`. Para cargas intensivas de CPU (*CPU-bound*), se emplea `concurrent.futures.ProcessPoolExecutor`, compilación sin GIL (PEP 703) o núcleos vectorizados en Rust/C mediante `PyO3`, `NumPy` o `PyTorch`. En metaprogramación, el módulo estándar `ast` permite inspeccionar, transformar y compilar árboles sintácticos de forma segura sin usar `eval()` arbitrario, técnica demostrada en nuestro proyecto verificado `projects/python-bytecode-vm`:

```
import ast

def count_binary_ops(expr_source: str) -> int:
    tree = ast.parse(expr_source, mode="eval")
    return sum(1 for node in ast.walk(tree) if isinstance(node, ast.BinOp))
```

Para registrar subclases o validar esquemas de clase en el momento de su creación, Python 3.6+ ofrece el gancho ligero `__init_subclass__(cls, **kwargs)`, evitando en el 95 % de los casos la complejidad de definir una metaclase personalizada que herede de `type`.

3.2 Bloques G y H: Ecosistema industrial (FastAPI, Pydantic, SQLAlchemy, Pytest) y los 4 proyectos (python-bytecode-vm)

El ecosistema profesional de Python (Bloque G) destaca por cuatro pilares: (1) desarrollo web y APIs con `FastAPI` (ASGI asíncrono), `Django` y validación de esquemas en Rust con `Pydantic v2`; (2) persistencia y migraciones con `SQLAlchemy 2.0` y `Alembic`; (3) ingeniería de datos y cálculo científico con `NumPy`, `SciPy`, `Pandas` y `Polars`; y (4) calidad y empaquetado con `pytest` (fixtures, parametrización), `Hypothesis` (property-based testing), `ruff`, `pyright` y construcción de paquetes `.whl` con `hatchling` o `maturin`. En el Bloque H, el lector desarrolla cuatro proyectos completos: un gestor CLI de tareas con persistencia SQLite, un cliente/servidor HTTP asíncrono con reintentos exponenciales, un pipeline de validación de datos y la máquina virtual de pila `projects/python-bytecode-vm`, verificada al 100 % con `pytest`, `ruff` y `pyright`.

Problema resuelto

Enunciado Un agregador de microservicios en Python debe consultar $N = 20$ endpoints HTTP independientes, cada uno con una latencia de red de $L = 150$ ms y 1 ms de procesamiento de CPU local en el bucle de eventos. Calcula cuántos milisegundos se ahorran al ejecutarlos concurrentemente con `asyncio.TaskGroup` ($150 + 20 \times 1 = 170$ ms) frente a ejecutarlos secuencialmente de forma síncrona ($20 \times (150 + 1) = 3020$ ms).
Datos

- $N = 20$ — Número de peticiones HTTP
- $L = 150\,\text{ms}$ — Latencia de red por petición
- $t_{cpu} = 1\,\text{ms}$ — Tiempo de CPU por respuesta

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Solapamiento de latencia de E/S en el bucle de eventos de asyncio

$$\Delta T = 20 \cdot 151 - (150 + 20)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos el tiempo concurrente (170 ms) al tiempo secuencial (3020 ms)

$$\Delta T = 20 \cdot 151 - (150 + 20)$$

$$((20 \cdot (150 + 1)) - (150 + (20 \cdot 1)))$$

Resultado

$$\textit{Resultado} = 2850$$

Interpretación La ejecución concurrente con asyncio ahorra 2850 ms, reduciendo el tiempo de respuesta total de 3.02 s a 0.17 s.
Comprobación de los pasos

- Paso 1: 2850 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0015] (Básico, Numérico) En `projects/python-bytecode-vm`, la expresión `2 * 3 + 4 * 5` emite 4 instrucciones `LOAD_CONST` y 3 instrucciones binarias (`MUL`, `MUL`, `ADD`). ¿Cuántas instrucciones totales contiene el bloque compilado?

2 [PRB-0016] (Básico, Numérico) Al evaluar `2 * 3 + 4 * 5` en una máquina de pila, ¿cuál es la altura máxima (número máximo de operandos apilados simultáneamente) que alcanza la pila?

3 [PRB-0017] (Intermedio, Desarrollo por pasos) Un pool de 8 procesos (`ProcessPoolExecutor(max_workers=8)`) ejecuta 40 tareas de CPU de 2 segundos cada una. ¿Cuántos segundos tarda en completar las 40 tareas ideales ($40/8 \times 2$)?

4 [PRB-0018] (Intermedio, Diseño) Una política de reintentos con retroceso exponencial (`backoff`) espera $100 \cdot 2^k$ ms para $k = 0, 1, 2, 3$. ¿Cuántos milisegundos totales habrá esperado acumulados tras esos 4 reintentos?

Parte II

Monografías Autónomas de JavaScript (ES2024+) y TypeScript (5.x)

Capítulo 4

Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM

*Este capítulo inicia la monografía autónoma de **JavaScript (ECMAScript 2024+)**, diferenciándolo con rigor de TypeScript. JavaScript es el único lenguaje nativo universal de los navegadores web y, a través de motores como V8 (Node.js, Deno) y JavaScriptCore (Bun), un entorno de servidor de altísima velocidad cuando se comprende cómo optimiza el compilador JIT sus estructuras dinámicas.*

Consejo

Para preservar el polimorfismo monomórfico en las *Inline Caches* de V8, inicializa siempre todas las propiedades de un objeto en su constructor en el mismo orden y evita eliminar propiedades dinámicamente con `delete obj.prop`.

Definición

Zona Muerta Temporal (Temporal Dead Zone, TDZ en ECMAScript): intervalo de ejecución desde la entrada al bloque léxico hasta la evaluación de la declaración `let` o `const` de una variable, durante el cual cualquier intento de leer o escribir dicha variable lanza determinísticamente un `ReferenceError`.

Autoficha

¿Por qué en JavaScript `0.1 + 0.2 === 0.3` evalúa a `false` y cómo deben representarse importes monetarios exactos usando `BigInt` en céntimos?

4.1 Bloques A–C: Historia de ECMAScript, motores JIT (V8), tipos, coerción vs ===, let/const y funciones

Creado por Brendan Eich en Netscape en 1995 y estandarizado por Ecma International bajo la especificación **ECMA-262**, JavaScript experimentó su transformación decisiva en ES6 (ES2015) con la introducción de `let/const`, funciones flecha, clases, promesas, generadores, `Map/Set`, `Symbol` y módulos nativos ESM. En los motores modernos como V8, el código fuente no se interpreta línea a línea: se compila a bytecode (*Ignition*) y las funciones calientes se optimizan a código máquina nativo (*TurboFan*) asociando a cada objeto una **clase oculta** (*Hidden Class* o *Shape/Map*). Si todas las instancias de un punto se inicializan con las mismas propiedades en el mismo orden (`{ x, y }`), comparten la misma Shape y los accesos `pt.x` se resuelven en una sola instrucción mediante *Inline Caching* (monomórfico). En cuanto a los tipos primitivos, `number` es un flotante IEEE 754 de doble precisión cuyos enteros exactos llegan hasta `Number.MAX_SAFE_INTEGER` ($2^{53} - 1 = 9007199254740991$), mientras que `BigInt` (`42n`) representa enteros de precisión arbitraria. En código profesional se utiliza siempre la igualdad estricta `===` y `!==` (que no realiza coerción implícita de tipos) o `Object.is(a, b)` (que distingue `+0` de `-0` y evalúa `Object.is(NaN, NaN)` a `true`).

4.2 Bloque D: Clausuras, delegación por prototipos, clases, desestructuración y módulos ES (ESM vs CommonJS)

A diferencia de los lenguajes basados en clases estáticas, el modelo de objetos de JavaScript se fundamenta en la **delegación prototípica**: cada objeto posee un enlace interno `[[Prototype]]` (accesible mediante `Object.getPrototypeOf(obj)`). Cuando se lee una propiedad `obj.metodo`, si no existe como

propiedad propia (`Object.hasOwn(obj, 'metodo')`), el motor asciende por la cadena de prototipos hasta encontrarla o llegar a `null`. La sintaxis `class` de ES2015/ES2022 (incluyendo campos privados reales `#saldo` verificados por la gramática) se construye limpiamente sobre este mecanismo:

```
export class TokenBucket {
  #capacity;
  #tokens;
  constructor(capacity) {
    this.#capacity = capacity;
    this.#tokens = capacity;
  }
  tryConsume(amount = 1) {
    if (amount <= 0 || this.#tokens < amount) return false;
    this.#tokens -= amount;
    return true;
  }
  get available() { return this.#tokens; }
}
```

Asimismo, mientras que las funciones tradicionales determinan el valor de `this` dinámicamente según el sitio de llamada (`obj.fn()`, `fn.call(ctx)`), las funciones flecha (`() => ...`) carecen de `this` propio y lo capturan léxicamente del ámbito circundante.

Problema resuelto

Enunciado En el motor V8 con compresión de punteros (*Pointer Compression*), cada referencia a propiedad en un objeto de forma fija (*Fast Properties / In-object properties*) ocupa 4 B en lugar de 8 B, más una cabecera de objeto de 12 B (`map`, `properties`, `elements` comprimidos). Calcula cuántos bytes ahorran $N = 10000$ objetos con $k = 5$ propiedades en línea bajo compresión de punteros ($12 + 5 \times 4 = 32$ B por objeto) frente a punteros de 64 bits sin comprimir ($24 + 5 \times 8 = 64$ B por objeto).

Datos

- $N = 10000$ — Número de objetos instanciados
- $S_{64} = 64\,\text{B}$ — Tamaño por objeto con punteros de 8 B (24 + 40)
- $S_{comp} = 32\,\text{B}$ — Tamaño por objeto con punteros comprimidos de 4 B (12 + 20)

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Ahorro de montículo mediante Pointer Compression en el motor V8

$$\Delta M = 10000 \cdot (64 - 32)$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 10000 objetos por los 32 B ahorrados en cada uno

$$\Delta M = 10000 \cdot (64 - 32)$$

$$(10000 \cdot ((24 + (5 \cdot 8)) - (12 + (5 \cdot 4))))$$

Resultado

$$\textit{Resultado} = 320000$$

Interpretación La compresión de punteros de V8 reduce el consumo a la mitad exacta, ahorrando 320 000 bytes.

Comprobación de los pasos

- Paso 1: 320000 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0021] (Básico, Numérico) En V8 con punteros comprimidos de 4 bytes, ¿cuántos bytes ocupa la cabecera de 3 campos (`map`, `properties`, `elements`) de un objeto JS (3×4)?
2 [PRB-0022] (Básico, Numérico) Un `ArrayBuffer(1024)` se observa mediante un `Float64Array` (8 bytes por elemento). ¿Cuántos elementos `length` tiene la vista tipada?
3 [PRB-0023] (Intermedio, Desarrollo por pasos) Si una cadena de prototipos tiene 4 niveles (`instancia -> SubClase.prototype -> Base.prototype -> Object.prototype -> null`) y se busca una propiedad inexistente, ¿cuántos eslabones de objeto se inspeccionan antes de llegar a `null`?
4 [PRB-0024] (Intermedio, Diseño) En `TokenBucket` con capacidad inicial de 100 tokens, se procesan 7 peticiones que consumen 12 tokens cada una. ¿Cuántos tokens quedan disponibles?

Capítulo 5

Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos

*Este capítulo completa la monografía de JavaScript cubriendo los Bloques E, F, G y H: el funcionamiento exacto del **Event Loop**, la diferencia crítica entre microtareas y macrotareas, el manejo de contrapresión (*backpressure*) en flujos de datos y el paralelismo real mediante `Workers`.*

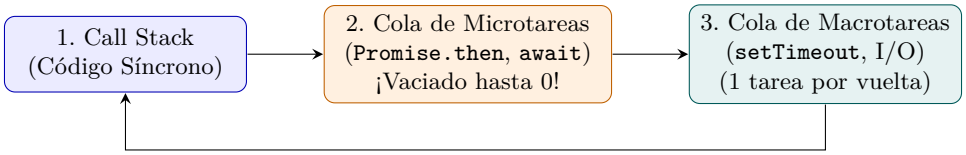


Figura 2. Ciclo determinista del Event Loop: tras vaciar la pila de llamadas síncrona (Call Stack), el motor drena exhaustivamente TODas las microtareas encoladas (incluidas las añadidas por otras microtareas) antes de tomar una única macrotarea.

Consejo

Cuando lances múltiples operaciones asíncronas independientes entre sí dentro de una función `async`, no escribas `const a = await fetchA(); const b = await fetchB();` en serie: lánzalas en paralelo con `const [a, b] = await Promise.all([fetchA(), fetchB()]);`.

Definición

Contrapresión (Backpressure en Streams): mecanismo de control de flujo mediante el cual un consumidor lento señala al productor asíncrono que su búfer interno ha alcanzado la marca de agua alta (`highWaterMark`), pausando la lectura de la fuente subyacente hasta que el búfer se drena.

Autoficha

¿Qué diferencia existe entre `Promise.all`, `Promise.allSettled`, `Promise.race` y `Promise.any` cuando una de las promesas de entrada se rechaza con un error?

5.1 Bloques E y F: Event Loop, cola de microtareas vs macrotareas, Promises, async/await, Streams y Workers

El modelo de concurrencia de JavaScript se basa en un hilo principal con ejecución hasta completitud (*run-to-completion*) gobernado por el **Event Loop**. Como muestra la Figura \ref{fig:fig-js-event-loop-queues}, existen dos colas con semánticas de vaciado radicalmente distintas: la **cola de microtareas** (*Jobs* en ECMA-262: continuaciones `.then/.catch/.finally` de `Promise`, código posterior a un `await` y `queueMicrotask`) y la **cola de tareas o macrotareas** (`setTimeout`, `setInterval`, eventos del DOM, callbacks de red/disco en `libuv`). Regla fundamental: cada vez que el *Call Stack* queda vacío, el motor ejecuta **todas** las microtareas pendientes hasta dejar la cola de microtareas completamente vacía antes de procesar la siguiente macrotarea o repintar la interfaz:

```
// Orden determinista de salida: [1, 4, 3, 2]
export function traceEventLoopOrder() {
  const log = [];
  log.push(1);
  setTimeout(() => log.push(2), 0);               // Macrotarea
```

```
    Promise.resolve().then(() => log.push(3));      // Microtarea
    log.push(4);
    return log;
}
```

Por consiguiente, un bucle recursivo infinito de `queueMicrotask` o `Promise.resolve().then(...)` hambrea por completo las macrotareas de E/S y congela el proceso aunque nunca bloquee mediante un `while(true)` síncrono.

5.2 Bloques G y H: Ecosistema (Node.js, Deno, Bun, npm/pnpm, Vitest, ESLint) y los 4 proyectos en JavaScript

En el Nivel Experto (Bloque F), cuando se procesan ficheros o respuestas HTTP de varios gigabytes en Node.js o la Web API, cargar todo el cuerpo en un `Buffer` agota el montículo de V8; en su lugar se utilizan `ReadableStream` / `WritableStream` con `pipeline()`, que regulan automáticamente la velocidad del productor según la capacidad del consumidor (*backpressure*). Para cómputo intensivo de CPU (criptografía, compresión, procesamiento de imágenes), se emplean `Worker Threads` (Node.js) o `Web Workers` (navegador) comunicados por paso de mensajes (`Structured Clone`), transferencia de propiedad de coste cero (`ArrayBuffer` transferibles) o memoria compartida sincronizada con `SharedArrayBuffer` y `Atomics`. El Bloque G cubre la gestión de paquetes con `pnpm`/`npm`, `package.json` (`exports` condicionales ESM), `ESLint`, `Prettier` y `Vitest` / `node -test`, y el Bloque H presenta cuatro proyectos completos en JavaScript puro (motor de plantillas reactivo sin dependencias, planificador de colas asíncrono con límite de concurrencia `p-limit`, procesador de logs por streams y servidor HTTP REST).

Problema resuelto

Enunciado Un descargador asíncrono en Node.js debe procesar $N = 120$ ficheros remotos con un limitador de concurrencia que mantiene exactamente $k = 15$ promesas en vuelo simultáneamente para no saturar el servidor de origen. Si cada descarga tarda $D = 200$ ms, calcula el tiempo total en milisegundos necesario para completar las 120 descargas $((120/15) \times 200)$.

Datos

- $N = 120$ — Total de tareas asíncronas
- $k = 15$ — Concurrencia máxima permitida
- $D = 200\,\text{ms}$ — Duración de cada descarga

Incógnita Magnitud objetivo solicitada en el enunciado

Modelo físico

Tiempo de finalización con concurrencia acotada (Semaphore / Pool de Promesas)

$$T = \frac{120}{15} \cdot 200$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Dividimos las 120 tareas en 8 oleadas de 15 y multiplicamos por 200 ms

$$T = \frac{120}{15} \cdot 200$$

$$\left(\frac{120}{15} \cdot 200\right)$$

Resultado

$$\textit{Resultado} = 1600$$

Interpretación Las 120 descargas finalizan en 1600 ms (1.6 segundos) manteniendo el uso de sockets acotado en 15.
Comprobación de los pasos

- Paso 1: 1600 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0027] (Básico, Numérico) En un turno del Event Loop se encolan 3 macrotareas (`setTimeout`) y 5 microtareas (`Promise.then`). ¿Cuántas microtareas se ejecutan antes de que comience la primera macrotarea?
2 [PRB-0028] (Básico, Numérico) Un `ReadableStream` con `highWaterMark = 65536` bytes (64 KiB) contiene actualmente 49 152 bytes sin consumir. ¿Cuántos bytes más puede aceptar antes de activar la señal de contrapresión?
3 [PRB-0029] (Intermedio, Desarrollo por pasos) Si 3 promesas independientes tardan 120 ms, 350 ms y 210 ms respectivamente, ¿en cuántos milisegundos se resuelve `Promise.all` de las tres?
4 [PRB-0030] (Intermedio, Diseño) En el problema anterior, si en lugar de `Promise.all` se ejecutaran secuencialmente con tres `await` uno tras otro, ¿cuántos milisegundos adicionales tardaría la operación ($120 + 350 + 210 - 350$)?

Capítulo 6

Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas

*Este capítulo inicia la monografía diferenciada de **TypeScript (5.x)** (Bloques A, B, C y D). Explicamos con rigor matemático cómo su sistema de tipos estructural modela toda la flexibilidad dinámica de JavaScript mediante tipos algebraicos y análisis de flujo de control, y por qué el **borrado de tipos** (*type erasure*) exige combinar tipos estáticos con validadores en las fronteras de red.*

Consejo

Activa siempre en `tsconfig.json` tanto `"strict": true` como `"noUncheckedIndexedAccess": true`: sin esta última opción, indexar un array `arr[i]` o un diccionario `dict[k]` se tipa como `T` en lugar de `T | undefined`, ocultando posibles accesos fuera de límites.

Definición

Subtipado Estructural por Anchura y Profundidad: relación de subtipado $S <: T$ en la que un registro $S = \{l_i : S_i\}_{i \in I}$ es subtipo de $T = \{l_j : T_j\}_{j \in J}$ si el conjunto de etiquetas de T está contenido en el de S ($J \subseteq I$, anchura) y para cada $j \in J$ inmutable se cumple $S_j <: T_j$ (profundidad).

Autoficha

¿Por qué TypeScript aplica una comprobación adicional de *propiedades en exceso* (*excess property checking*) cuando se pasa un literal de objeto directamente a una función, pero la omite si ese mismo objeto se guarda primero en una variable intermedia?

6.1 Bloques A–C: Filosofía de TypeScript, borrado de tipos, tsconfig.json estricto y primitivos (unknown, never)

Creado por Anders Hejlsberg en Microsoft y publicado en 2012, TypeScript es un superconjunto sintáctico estáticamente tipado de ECMAScript. Su principio arquitectónico fundamental es el **borrado de tipos** (*type erasure*): una vez que el comprobador de tipos (`tsc`) verifica la ausencia de errores estáticos, todas las anotaciones de tipos, interfaces, alias `type` y parámetros genéricos desaparecen por completo al emitir JavaScript (o al ejecutar directamente el fichero `.ts` mediante `node -experimental-strip-types` en Node.js 22+/24+). En consecuencia, las anotaciones de TypeScript tienen **coste cero** en tiempo de ejecución, pero tampoco realizan validaciones automáticas sobre un JSON recibido de la red (`JSON.parse` devuelve datos cuya forma real debe verificarse en tiempo de ejecución antes de estrechar su tipo). En teoría de tipos, `unknown` es el tipo superior seguro (*top type*: acepta cualquier valor pero prohíbe operar sobre él sin estrecharlo primero), `never` es el tipo inferior vacío (*bottom type*, conjunto $\emptyset$: habita las funciones que nunca retornan y las ramas inalcanzables), mientras que `any` desactiva el sistema de tipos y debe prohibirse mediante `@typescript-eslint/no-explicit-any`.

6.2 Bloque D: Tipado estructural, interfaces vs type aliases, estrechamiento (narrowing) y uniones discriminadas

A diferencia de Java o C#, TypeScript utiliza **subtipado estructural** (*structural subtyping*): un tipo S es subtipo de T ($S <: T$) si y solo si S posee al menos todas las propiedades requeridas por T con tipos compatibles, sin importar cómo se declararon sus nombres. La herramienta de modelado de

dominio más potente de TypeScript es la **unión discriminada** (*discriminated union*): una unión de tipos objeto que comparten una propiedad literal discriminante común (como `kind` o `ok`). Como se implementa en nuestro proyecto verificado `projects/ts-reactive-http`, el compilador estrecha (*narrows*) automáticamente el tipo en cada rama condicional y verifica la exhaustividad asignando el caso residual al tipo vacío `never`:

```
export type DomainEvent =
  | { readonly kind: "created"; readonly id: string; readonly timestamp: number }
  | { readonly kind: "paid"; readonly id: string; readonly amountCents: number }
  | { readonly kind: "cancelled"; readonly id: string; readonly reason: string };

export function formatEventSummary(ev: DomainEvent): string {
  switch (ev.kind) {
    case "created":   return `Created ${ev.id} at ${ev.timestamp}`;
    case "paid":      return `Paid ${ev.id}: ${ev.amountCents} cents`;
    case "cancelled": return `Cancelled ${ev.id}: ${ev.reason}`;
    default: {
      const _exhaustive: never = ev;
      return _exhaustive;
    }
  }
}
```

Si en el futuro un ingeniero añade una cuarta variante a `DomainEvent`, el bloque `default` fallará en compilación porque la nueva variante no es asignable a `never`.

Problema resuelto

Enunciado En un módulo de pagos en TypeScript, un estado mal modelado con banderas opcionales independientes `{ status: 'idle' | 'loading' | 'success' | 'error'; isRetry: boolean; hasCache: boolean; isOffline: boolean }` tiene $4 \times 2 \times 2 \times 2 = 32$ combinaciones posibles. Al refactorizarlo en una unión discriminada exacta donde solo existen 9 estados válidos de dominio, ¿cuántos estados inválidos se vuelven estáticamente irrepresentables en compilación?
Datos

- $C_{prod} = 32$ — Estados del tipo producto cartesiano $(4 * 2^3)$
- $C_{suma} = 9$ — Estados legítimos de la unión discriminada

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Reducción de cardinalidad de estados mediante uniones discriminadas en TypeScript

$$\Delta C = 4 \cdot 2^3 - 9$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Restamos los 9 estados legítimos a las 32 combinaciones del producto cartesiano

$$\Delta C = 4 \cdot 2^3 - 9$$

$$((((4 \cdot 2) \cdot 2) \cdot 2) - 9)$$

Resultado

$$Resultado = 23$$

Interpretación Se eliminan de raíz 23 estados imposibles o incoherentes sin escribir una sola prueba adicional.
Comprobación de los pasos

- Paso 1: 23 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0033**] **(Básico, Numérico)** Un tipo literal unión `type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH'` se combina en una tupla `[HttpMethod, boolean]`. ¿Cuántos valores distintos habitan esa tupla (5×2)?
2 [**PRB-0034**] **(Básico, Numérico)** ¿Cuántos bytes adicionales ocupa en el fichero `.js` emitido por `tsc` una interfaz `interface UserProfile` de 45 líneas tras el borrado de tipos?
3 [**PRB-0035**] **(Intermedio, Desarrollo por pasos)** Una unión discriminada tiene 6 variantes y un `switch` cubre actualmente 4 de ellas. ¿Cuántas variantes quedan en el tipo estrechado de la variable al llegar a la rama `default`?
4 [**PRB-0036**] **(Intermedio, Diseño)** En `projects/ts-reactive-http`, el limitador de tasa se inicializa con capacidad 3 y se envían 4 peticiones válidas seguidas. ¿Cuántas respuestas retornan con código HTTP `201` antes de que salte el código `429`?

Capítulo 7

Monografía de TypeScript (II – Bloques E–H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos

Este capítulo lleva al lector a los niveles Avanzado y Experto de TypeScript (Bloques E, F, G y H). El sistema de tipos de TypeScript constituye por sí mismo un lenguaje funcional puro que se evalúa en tiempo de compilación sobre conjuntos de tipos, permitiendo calcular tipos de retorno, rutas tipadas y esquemas recursivos.

Consejo

Utiliza el operador `satisfies` para diccionarios de configuración y tablas de rutas: `const routes = { ... } satisfies Record<string, RouteConfig>` comprueba cada entrada contra `RouteConfig` pero conserva las claves literales exactas en `keyof typeof routes`.

Definición

Distributividad de Tipos Condicionales en TypeScript: propiedad según la cual, para un tipo condicional `type F<T> = T extends U ? X : Y` donde `T` es un parámetro genérico desnudo, instanciar `F<A | B>` se distribuye automáticamente como `F<A> | F<B>` (pudiendo desactivarse escribiendo `[T] extends [U] ? X : Y`).

Autoficha

¿Por qué `Exclude<'a' | 'b' | 'c', 'a'>` devuelve `'b' | 'c'` gracias a la distributividad de `T extends U ? never : T` y por qué `never` actúa como el elemento neutro de la unión (`T | never = T`)?

7.1 Bloques E y F: Generics, keyof, Mapped Types, Conditional Types, infer, Template Literals, satisfies y Branded Types

La programación a nivel de tipos en TypeScript se apoya en cuatro primitivas composicionales: (1) el operador de consulta de claves `keyof T` y el acceso indexado `T[K]`; (2) los **tipos mapeados** (`{ readonly [K in keyof T as ...]: ... }`), sobre los que se construyen utilidades estándar como `Partial<T>`, `Required<T>`, `Readonly<T>`, `Pick<T, K>` y `Record<K, V>`; (3) los **tipos condicionales** `T extends U ? X : Y`, que se distribuyen automáticamente sobre uniones cuando `T` es un parámetro de tipo desnudo y permiten extraer tipos internos mediante `infer` (como `type AwaitedValue<T> = T extends Promise<infer U> ? AwaitedValue<U> : T`); y (4) los **Template Literal Types**, que calculan productos cartesianos de cadenas en compilación. Además, TypeScript 4.9+ introdujo el operador `satisfies`, que valida que una expresión cumpla un contrato `T` **sin ensanchar** (*widening*) su tipo literal inferido, y para los casos donde el tipado estructural resulta demasiado permisivo (por ejemplo, evitar pasar un `UserId` donde se espera un `OrderId`, siendo ambos `string`), se emplean **Branded Types**:

```
declare const __brand: unique symbol;
export type Brand<T, B extends string> = T & { readonly [__brand]: B };

export type UserId  = Brand<string, "UserId">;
export type OrderId = Brand<string, "OrderId">;
export type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
```

```
};
```

Con `Brand<string, "UserId">`, la marca fantasma `unique symbol` existe únicamente en el comprobador de tipos y se borra por completo en ejecución, proporcionando seguridad nominal con coste cero.

7.2 Bloques G y H: Ecosistema (Zod, tRPC, Vite, tsx, Vitest) y los 4 proyectos (incluyendo ts-reactive-http)

En el Bloque G (Ecosistema), la práctica estándar de la industria combina TypeScript con validadores de esquema en las fronteras de entrada/salida (como `Zod` o `Valibot`, donde `type Payload = z.infer<typeof PayloadSchema>` deriva el tipo estático directamente del validador de ejecución para que jamás diverjan), comunicación extremo a extremo con seguridad de tipos (`tRPC`, OpenAPI/GraphQL codegen), empaquetadores rápidos (`Vite`, `esbuild`, `tsup`) y ejecución directa en Node.js (`node -experimental-strip-types` o `tsx`). En el Bloque H, junto con un cliente HTTP tipado por rutas, un motor de máquinas de estados finitos (`StateMachine<S, E>`) y un bus de eventos tipado, estudiamos el proyecto completo `projects/ts-reactive-http`, verificado con `node -experimental-strip-types -test`.

Problema resuelto

Enunciado En un sistema de diseño tipado en TypeScript, se define el tipo de clases utilitarias mediante Template Literal Types: `type SpacingUtility =` ${Direction}-${Axis}-${Scale}`, donde` Direction `tiene $2$ miembros ('m' | 'p'),` Axis `tiene $7$ miembros ('t' | 'b' | 'l' | 'r' | 'x' | 'y' | 'a') y` Scale `tiene $12$ miembros ('0' a '11'). Calcula cuántos tipos literales de cadena exactos genera el compilador en la unión` SpacingUtility`.`
Datos

- $Direction = 2$ — Margen o padding ('m' | 'p')
- $Axis = 7$ — Ejes direccionales
- $Scale = 12$ — Niveles de la escala espacial

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Producto cartesiano en Template Literal Types de TypeScript

$$|U| = 2 \cdot 7 \cdot 12$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos las cardinalidades de las tres uniones literales

$$|U| = 2 \cdot 7 \cdot 12$$

$$((2 \cdot 7) \cdot 12)$$

Resultado

$$Resultado = 168$$

Interpretación El compilador genera y autocompleta exactamente 168 literales de cadena válidos en tiempo de compilación.
Comprobación de los pasos

- Paso 1: 168 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [**PRB-0039**] **(Básico, Numérico)** Una interfaz `Config` tiene 8 propiedades obligatorias. Si se aplica `Pick<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades tiene el tipo resultante?
2 [**PRB-0040**] **(Básico, Numérico)** En el problema anterior, si en su lugar se aplica `Omit<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades conserva el tipo resultante?
3 [**PRB-0041**] **(Intermedio, Desarrollo por pasos)** Un Template Literal Type `${'http' | 'https'}://${'api' | 'auth' | 'cdn'}.ejemplo.es/${'v1' | 'v2'}` expande todas las rutas válidas. ¿Cuántos literales de URL compone la unión?
4 [**PRB-0042**] **(Intermedio, Diseño)** En `projects/ts-reactive-http`, cada lectura de telemetría se redondea con `Math.round(v * 100) / 100`. Para el vector `[12.5, 7.25, 10.25]`, ¿cuál es el valor exacto de la suma?

Capítulo 8

Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript

Este capítulo de síntesis compara con rigor científico los tres lenguajes del Volumen V y muestra cómo combinarlos en arquitecturas reales de producción (por ejemplo, un frontend y pasarela API en TypeScript comunicándose con un motor de cálculo científico o de aprendizaje automático en Python).

Consejo

En arquitecturas políglotas Python + TypeScript, utiliza el esquema OpenAPI exportado por FastAPI/Pydantic o definiciones Protobuf como única fuente de verdad en el pipeline de CI para detectar rupturas de contrato antes de desplegar.

Definición

Ejecución ansiosa frente a perezosa de tareas asíncronas (Eager Promises vs Lazy Coroutines): en ECMAScript el ejecutor de `new Promise(fn)` se invoca síncronamente al instanciarse, mientras que en Python (y Rust) invocar una función `async` devuelve un objeto corrutina inerte hasta que el bucle de eventos lo impulsa.

Autoficha

¿Por qué llamar a una función `async def f()` en Python sin anteponer `await` ni `asyncio.create_task(f())` no ejecuta ni una sola línea del cuerpo de `f` y emite un `RuntimeWarning: coroutine 'f' was never awaited`?

8.1 Comparativa profunda de ejecución, memoria, concurrencia y sistemas de tipos (Python vs JS vs TS)

Aunque Python y JavaScript comparten tipado dinámico en tiempo de ejecución y gestión automática de memoria, sus decisiones de diseño divergen en aspectos esenciales. Primero, en **semántica numérica y de tipos**: Python posee enteros de precisión arbitraria (`int`) y distingue estrictamente entre tipos sin coerción implícita entre cadenas y números (`"5" + 1` lanza `TypeError`), mientras que JavaScript unifica números en `Number` (IEEE 754) + `BigInt` y realiza coerciones implícitas si no se usa `===` o TypeScript. Segundo, en **modelo de objetos**: Python se basa en clases reales donde incluso las metaclases son objetos (`type`), mientras que JavaScript se basa en delegación de prototipos (`[[Prototype]]`). Tercero, en **asincronía**: en JavaScript una `Promise` es *ansiosa* (*eager*, comienza a ejecutarse en el instante en que se construye y el Event Loop está siempre activo por defecto), mientras que una corrutina `async def` en Python es estrictamente *perezosa* hasta que es agendada o esperada dentro de `asyncio.run(...)`.

8.2 Arquitectura políglota: contratos OpenAPI/JSON Schema entre Python y TypeScript y evaluación global

Tanto el sistema de anotaciones de Python (`typing`, PEP 484/695) como TypeScript aplican borrado de tipos en tiempo de ejecución (*type erasure*). Sin embargo, dado que en Python las anotaciones pueden conservarse opcionalmente en el diccionario `__annotations__` de la clase o función, bibliotecas como `Pydantic` y `FastAPI` inspeccionan dichas anotaciones al arrancar el proceso para compilar validadores en Rust y emitir automáticamente un documento **OpenAPI 3.1 / JSON Schema**. A partir de ese

único contrato normativo, el equipo de TypeScript genera sus tipos estáticos y clientes HTTP sin duplicar definiciones manualmente, garantizando consistencia matemática entre ambos extremos del sistema distribuido.

Problema resuelto

Enunciado Una pasarela en TypeScript envía lotes de $N = 5000$ eventos de telemetría a un servicio de análisis en Python. En formato JSON textual cada objeto `{ "sensorId": ..., "timestamp": ..., "value": ... }` ocupa en promedio $S_{\text{json}} = 76$ B, mientras que codificado en binario compacto ocupa $S_{\text{bin}} = 20$ B. Calcula cuántos kilobytes (dividiendo entre 1000) se ahorran por cada lote transmitido.
Datos

- $N = 5000$ — Eventos por lote
- $S_{json} = 76\,\text{B}$ — Tamaño medio en JSON UTF-8
- $S_{bin} = 20\,\text{B}$ — Tamaño en binario compacto

Incógnita Magnitud objetivo solicitada en el enunciado
Modelo físico
Ahorro de tráfico de red entre representación JSON y binaria

$$\Delta B_{\text{kB}} = \frac{5000 \cdot (76 - 20)}{1000}$$

Hipótesis

- Evaluación determinista bajo el modelo formal del capítulo

Desarrollo

1. Multiplicamos los 5000 eventos por los 56 B ahorrados y dividimos entre 1000 B/kB

$$\Delta B_{\text{kB}} = \frac{5000 \cdot (76 - 20)}{1000}$$

$$\frac{(5000 \cdot (76 - 20))}{1000}$$

Resultado

$$Resultado = 280$$

Interpretación Se ahorran 280 kB por lote (pasando de 380 kB a 100 kB).
Comprobación de los pasos

- Paso 1: 280 — reproducido

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicios

1 [PRB-0045] (Básico, Numérico) Un contrato OpenAPI define 18 endpoints REST y cada endpoint especifica 3 esquemas (petición, respuesta 200 y error 400). ¿Cuántos esquemas genera automáticamente el generador de tipos de TypeScript (18×3)?
2 [PRB-0046] (Básico, Numérico) Si una respuesta JSON de 120 000 bytes se comprime con Brotli a un 25 % de su tamaño original ($120000 / 4$), ¿cuántos bytes se transmiten por la red?
3 [PRB-0047] (Intermedio, Desarrollo por pasos) Un servicio en Python delega un cálculo matricial a una extensión compilada en Rust (`PyO3`) reduciendo el tiempo por petición de 45 ms a 3 ms. Para 200 peticiones secuenciales, ¿cuántos milisegundos totales se ahorran?

4 [PRB-0048] (Intermedio, Diseño) Una aplicación web carga un módulo WebAssembly de 180 KiB y un bundle TypeScript/JS de 95 KiB. ¿Cuántos KiB totales descarga el cliente en el arranque inicial?

Apéndice A

Notación

Símbolo	Significado	Unidades
T extends $U ? X : Y$	Tipo condicional distributivo en el sistema de tipos de TypeScript	tipo
$\tau_{\text{micro}}, \tau_{\text{macro}}$	Colas de microtareas (Promises) y macrotareas (timers/I/O) del Event Loop	tareas
$\text{refcnt}(o)$	Contador de referencias de un PyObject en CPython	referencias

Apéndice B

Hoja de fórmulas

Esta parte del borrador aún no tiene material.

Apéndice C

Soluciones

C.1 Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante

Problema resuelto [PRB-0001]
En CPython de 64 bits, cada entrada de un diccionario almacena tres palabras de 8 B (`me_hash`, `me_key`, `me_value` = 24 B). Para una tabla con $K = 128$ ranuras en el índice (1 B por ranura en `dk_indices`) y capacidad para $N = 85$ entradas en `dk_entries`, calcula cuántos bytes ahorra el diseño de diccionario compacto ($128 \cdot 1 + 85 \cdot 24$) frente al diseño disperso antiguo ($128 \cdot 24$).
Respuesta

$$\textit{Resultado} = 904$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0003]
En CPython de 64 bits, la cabecera base `PyObject` contiene `ob_refcnt` (8 B) y `ob_type` (8 B). ¿Cuántos bytes mide esta cabecera mínima?
Respuesta 16
Ejercicio 2 [PRB-0004]
Una lista `PyListObject` mantiene un array de punteros `PyObject*` (8 B cada uno). Si su capacidad reservada es de 64 elementos, ¿cuántos bytes ocupa ese array de punteros?
Respuesta 512
Ejercicio 3 [PRB-0005]
¿Cuál es la suma de los cuadrados impares evaluada por `sum(x * x for x in range(1, 10, 2))` ($1^2 + 3^2 + 5^2 + 7^2 + 9^2$)?
Respuesta 165
Ejercicio 4 [PRB-0006]
En el proyecto `projects/python-bytecode-vm`, la función compila y evalúa `(x + 4) * (y - 2) + (10 if x < y else 20)` para $x = 6, y = 9$. ¿Cuál es el resultado entero devuelto?
Respuesta 80

Cálculos reproducidos

$((128 \cdot 24) - ((128 \cdot 1) + (85 \cdot 24))) = 904$

C.2 Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers

Problema resuelto [PRB-0007]
En una aplicación de procesamiento de grafos en CPython de 64 bits, una clase `Edge` con $k = 3$ atributos ocupa $16 + 3 \times 8 = 40$ B cuando se declara con `__slots__`, mientras que sin `__slots__` cada instancia aloja además un `__dict__` que añade 112 B por objeto (152 B en total). Calcula cuántos megabytes exactos (dividiendo entre 10^6) se ahorran al instanciar $N = 1000000$ de aristas con `__slots__`.
Respuesta

$$\textit{Resultado} = 112$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.

Ejercicio 1 [PRB-0009]
En CPython de 64 bits, ¿cuántos bytes ocupa una instancia con `__slots__` de 4 atributos (16 B de cabecera más 4×8 B)?
Respuesta 48
Ejercicio 2 [PRB-0010]
Un generador perezoso produce bloques de 4096 bytes uno a uno para procesar un fichero de 10 GiB. ¿Cuántos bloques mantiene simultáneamente en RAM?
Respuesta 1
Ejercicio 3 [PRB-0011]
En una jerarquía con herencia múltiple `class D(B, C)` donde `B(A)` y `C(A)` heredan de `A(object)`, ¿cuántas clases aparecen en la tupla de orden de resolución de métodos `D.__mro__` (`D, B, C, A, object`)?
Respuesta 5
Ejercicio 4 [PRB-0012]
Un decorador `@lru_cache(maxsize=256)` recibe 1000 consultas sobre un conjunto de solo 200 claves distintas. ¿Cuántas veces se ejecuta realmente la función subyacente?
Respuesta 200

Cálculos reproducidos

$\frac{(1000000 \cdot (152-40))}{1000000} = 112$

C.3 Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos

Problema resuelto [PRB-0013]
Un agregador de microservicios en Python debe consultar $N = 20$ endpoints HTTP independientes, cada uno con una latencia de red de $L = 150$ ms y 1 ms de procesamiento de CPU local en el bucle de eventos. Calcula cuántos milisegundos se ahorran al ejecutarlos concurrentemente con `asyncio.TaskGroup` ($150 + 20 \times 1 = 170$ ms) frente a ejecutarlos secuencialmente de forma síncrona ($20 \times (150 + 1) = 3020$ ms).
Respuesta

$$Resultado = 2850$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0015]
En `projects/python-bytecode-vm`, la expresión `2 * 3 + 4 * 5` emite 4 instrucciones `LOAD_CONST` y 3 instrucciones binarias (`MUL`, `MUL`, `ADD`). ¿Cuántas instrucciones totales contiene el bloque compilado?
Respuesta 7
Ejercicio 2 [PRB-0016]
Al evaluar `2 * 3 + 4 * 5` en una máquina de pila, ¿cuál es la altura máxima (número máximo de operandos apilados simultáneamente) que alcanza la pila?
Respuesta 3
Ejercicio 3 [PRB-0017]
Un pool de 8 procesos (`ProcessPoolExecutor(max_workers=8)`) ejecuta 40 tareas de CPU de 2 segundos cada una. ¿Cuántos segundos tarda en completar las 40 tareas ideales ($40/8 \times 2$)?
Respuesta 10
Ejercicio 4 [PRB-0018]
Una política de reintentos con retroceso exponencial (`backoff`) espera $100 \cdot 2^k$ ms para $k = 0, 1, 2, 3$. ¿Cuántos milisegundos totales habrá esperado acumulados tras esos 4 reintentos?
Respuesta 1500

Cálculos reproducidos

$((20 \cdot (150 + 1)) - (150 + (20 \cdot 1))) = 2850$

C.4 Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM

Problema resuelto [PRB-0019]
En el motor V8 con compresión de punteros (*Pointer Compression*), cada referencia a propiedad en un objeto de forma fija (*Fast Properties / In-object properties*) ocupa 4 B en lugar de 8 B, más una cabecera de objeto de 12 B (`map`, `properties`, `elements` comprimidos). Calcula cuántos bytes ahorran $N = 10000$ objetos con $k = 5$ propiedades en línea bajo compresión de punteros ($12 + 5 \times 4 = 32$ B por objeto) frente a punteros de 64 bits sin comprimir ($24 + 5 \times 8 = 64$ B por objeto).
Respuesta

$$\textit{Resultado} = 320000$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0021]
En V8 con punteros comprimidos de 4 bytes, ¿cuántos bytes ocupa la cabecera de 3 campos (`map`, `properties`, `elements`) de un objeto JS (3×4)?
Respuesta 12
Ejercicio 2 [PRB-0022]
Un `ArrayBuffer(1024)` se observa mediante un `Float64Array` (8 bytes por elemento). ¿Cuántos elementos `length` tiene la vista tipada?
Respuesta 128
Ejercicio 3 [PRB-0023]
Si una cadena de prototipos tiene 4 niveles (`instancia -> SubClase.prototype -> Base.prototype -> Object.prototype -> null`) y se busca una propiedad inexistente, ¿cuántos eslabones de objeto se inspeccionan antes de llegar a `null`?
Respuesta 4
Ejercicio 4 [PRB-0024]
En `TokenBucket` con capacidad inicial de 100 tokens, se procesan 7 peticiones que consumen 12 tokens cada una. ¿Cuántos tokens quedan disponibles?
Respuesta 16

Cálculos reproducidos

$(10000 \cdot ((24 + (5 \cdot 8)) - (12 + (5 \cdot 4)))) = 320000$

C.5 Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos

Problema resuelto [PRB-0025]
Un descargador asíncrono en Node.js debe procesar $N = 120$ ficheros remotos con un limitador de concurrencia que mantiene exactamente $k = 15$ promesas en vuelo simultáneamente para no saturar el servidor de origen. Si cada descarga tarda $D = 200$ ms, calcula el tiempo total en milisegundos necesario para completar las 120 descargas ($(120/15) \times 200$).
Respuesta

$$\textit{Resultado} = 1600$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0027]
En un turno del Event Loop se encolan 3 macrotareas (`setTimeout`) y 5 microtareas (`Promise.then`). ¿Cuántas microtareas se ejecutan antes de que comience la primera macrotarea?
Respuesta 5
Ejercicio 2 [PRB-0028]

Un `ReadableStream` con `highWaterMark = 65536` bytes (64 KiB) contiene actualmente 49 152 bytes sin consumir. ¿Cuántos bytes más puede aceptar antes de activar la señal de contrapresión?
Respuesta 16384
Ejercicio 3 [PRB-0029]
Si 3 promesas independientes tardan 120 ms, 350 ms y 210 ms respectivamente, ¿en cuántos milisegundos se resuelve `Promise.all` de las tres?
Respuesta 350
Ejercicio 4 [PRB-0030]
En el problema anterior, si en lugar de `Promise.all` se ejecutaran secuencialmente con tres `await` uno tras otro, ¿cuántos milisegundos adicionales tardaría la operación (\$120 + 350 + 210 - 350\$)?
Respuesta 330

Cálculos reproducidos

$\left(\frac{120}{15} \cdot 200\right) = 1600$

C.6 Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas

Problema resuelto [PRB-0031]
En un módulo de pagos en TypeScript, un estado mal modelado con banderas opcionales independientes `{ status: 'idle' | 'loading' | 'success' | 'error'; isRetry: boolean; hasCache: boolean; isOffline: boolean }` tiene $4 \times 2 \times 2 \times 2 = 32$ combinaciones posibles. Al refactorizarlo en una unión discriminada exacta donde solo existen 9 estados válidos de dominio, ¿cuántos estados inválidos se vuelven estáticamente irrepresentables en compilación?
Respuesta

$$Resultado = 23$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0033]
Un tipo literal unión `type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE' | 'PATCH'` se combina en una tupla `[HttpMethod, boolean]`. ¿Cuántos valores distintos habitan esa tupla (5×2)?
Respuesta 10
Ejercicio 2 [PRB-0034]
¿Cuántos bytes adicionales ocupa en el fichero `.js` emitido por `tsc` una interfaz `interface UserProfile` de 45 líneas tras el borrado de tipos?
Respuesta 0
Ejercicio 3 [PRB-0035]
Una unión discriminada tiene 6 variantes y un `switch` cubre actualmente 4 de ellas. ¿Cuántas variantes quedan en el tipo estrechado de la variable al llegar a la rama `default`?
Respuesta 2
Ejercicio 4 [PRB-0036]
En `projects/ts-reactive-http`, el limitador de tasa se inicializa con capacidad 3 y se envían 4 peticiones válidas seguidas. ¿Cuántas respuestas retornan con código HTTP `201` antes de que salte el código `429`?
Respuesta 3

Cálculos reproducidos

$((((4 \cdot 2) \cdot 2) \cdot 2) - 9) = 23$

C.7 Monografía de TypeScript (II – Bloques E–H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos

Problema resuelto [PRB-0037]
En un sistema de diseño tipado en TypeScript, se define el tipo de clases utilitarias mediante Template Literal Types: `type SpacingUtility = ${Direction}-${Axis}-${Scale}`, donde Direction tiene 2 miembros ('m' | 'p'), Axis tiene 7 miembros ('t' | 'b' | 'l' | 'r' | 'x' | 'y' | 'a') y Scale tiene 12 miembros ('0' a '11'). Calcula cuántos tipos literales de cadena exactos genera el compilador en la unión SpacingUtility`.
Respuesta

$$\textit{Resultado} = 168$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0039]
Una interfaz `Config` tiene 8 propiedades obligatorias. Si se aplica `Pick<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades tiene el tipo resultante?
Respuesta 3
Ejercicio 2 [PRB-0040]
En el problema anterior, si en su lugar se aplica `Omit<Config, 'host' | 'port' | 'tls'>`, ¿cuántas propiedades conserva el tipo resultante?
Respuesta 5
Ejercicio 3 [PRB-0041]
Un Template Literal Type `${'http' | 'https'}://${'api' | 'auth' | 'cdn'}.ejemplo.es/${'v1' | 'v2'}` expande todas las rutas válidas. ¿Cuántos literales de URL compone la unión?
Respuesta 12
Ejercicio 4 [PRB-0042]
En `projects/ts-reactive-http`, cada lectura de telemetría se redondea con `Math.round(v * 100) / 100`. Para el vector `[12.5, 7.25, 10.25]`, ¿cuál es el valor exacto de la suma?
Respuesta 30

Cálculos reproducidos

$((2 \cdot 7) \cdot 12) = 168$

C.8 Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript

Problema resuelto [PRB-0043]
Una pasarela en TypeScript envía lotes de $N = 5000$ eventos de telemetría a un servicio de análisis en Python. En formato JSON textual cada objeto `{ "sensorId": ..., "timestamp": ..., "value": ... }` ocupa en promedio $S_{\text{json}} = 76$ B, mientras que codificado en binario compacto ocupa $S_{\text{bin}} = 20$ B. Calcula cuántos kilobytes (dividiendo entre 1000) se ahorran por cada lote transmitido.
Respuesta

$$\textit{Resultado} = 280$$

Comprobación de la respuesta la aritmética del último paso reproduce la respuesta enunciada a la precisión con que está escrita. Es una reproducción aritmética; no valida el modelo físico ni sus hipótesis.
Ejercicio 1 [PRB-0045]
Un contrato OpenAPI define 18 endpoints REST y cada endpoint especifica 3 esquemas (petición, respuesta 200 y error 400). ¿Cuántos esquemas genera automáticamente el generador de tipos de TypeScript (18×3)?
Respuesta 54

Ejercicio 2 [PRB-0046]
Si una respuesta JSON de 120 000 bytes se comprime con Brotli a un 25 % de su tamaño original ($120000 / 4$), ¿cuántos bytes se transmiten por la red?
Respuesta 30000
Ejercicio 3 [PRB-0047]
Un servicio en Python delega un cálculo matricial a una extensión compilada en Rust (`PyO3`) reduciendo el tiempo por petición de 45 ms a 3 ms. Para 200 peticiones secuenciales, ¿cuántos milisegundos totales se ahorran?
Respuesta 8400
Ejercicio 4 [PRB-0048]
Una aplicación web carga un módulo WebAssembly de 180 KiB y un bundle TypeScript/JS de 95 KiB. ¿Cuántos KiB totales descarga el cliente en el arranque inicial?
Respuesta 275

Cálculos reproducidos

$\frac{(5000 \cdot (76 - 20))}{1000} = 280$

Apéndice D

Auditoría de fuentes

Estado de las fuentes: PENDING. 32 pendientes. Los conflictos almacenados quedan fuera (0). Las fuentes que la guía almacenada no cita quedan fuera. Recuento de no citadas: 0.
¶1: dos testimonios.
¶2: dos testimonios.
¶3: dos testimonios.
¶4: dos testimonios.
¶5: dos testimonios.
¶6: dos testimonios.
¶7: dos testimonios.
¶8: dos testimonios.
¶9: dos testimonios.
¶10: dos testimonios.
¶11: dos testimonios.
¶12: dos testimonios.
¶13: dos testimonios.
¶14: dos testimonios.
¶15: dos testimonios.
¶16: dos testimonios.
¶17: dos testimonios.
¶18: dos testimonios.
¶19: dos testimonios.
¶20: dos testimonios. Ecuaciones: 1 reproducida numéricamente.
¶21: dos testimonios.
¶22: dos testimonios.
¶23: dos testimonios.
¶24: dos testimonios.
¶25: dos testimonios.
¶26: dos testimonios.
¶27: dos testimonios.
¶28: dos testimonios.
¶29: dos testimonios.
¶30: dos testimonios.
¶31: dos testimonios.
¶32: dos testimonios.
¶33: dos testimonios.
¶34: dos testimonios. Ecuaciones: 2 valor o condición.
¶35: dos testimonios.
¶36: dos testimonios.
¶37: dos testimonios.
¶38: dos testimonios.
¶39: dos testimonios.
¶40: dos testimonios.
¶41: dos testimonios.
¶42: dos testimonios.
¶43: dos testimonios.
¶44: dos testimonios.
¶45: dos testimonios.
¶46: dos testimonios.
¶47: dos testimonios.
¶48: dos testimonios.

Referencia Normativa src-python-ref (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6, ¶7, ¶8, ¶9, ¶10, ¶11, ¶12, ¶13, ¶14, ¶15, ¶16, ¶17, ¶18, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-unicode-15 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶1, ¶2, ¶3, ¶4, ¶5, ¶6.
Referencia Normativa src-gamma-gof (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶7, ¶8, ¶9, ¶10, ¶11, ¶12.
Referencia Normativa src-kleppmann-ddia (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶13, ¶14, ¶15, ¶16, ¶17, ¶18.
Referencia Normativa src-ecmascript-2024 (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
Referencia Normativa src-mdn-html-css (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶19, ¶20, ¶21, ¶22, ¶23, ¶24, ¶25, ¶26, ¶27, ¶28, ¶29, ¶30.
The TypeScript Handbook — Structural Typing, Narrowing, and Type Manipulation (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42, ¶43, ¶44, ¶45, ¶46, ¶47, ¶48.
Referencia Normativa src-pierce-tapl (PENDING) — Suplemento abierto: — No es la bibliografía de la guía. — ¶31, ¶32, ¶33, ¶34, ¶35, ¶36, ¶37, ¶38, ¶39, ¶40, ¶41, ¶42.
REV-0001: cap01-python-fundamentos-modelo-objetos-nivel-principiante scientific
REV-0002: cap02-python-intermedio-avanzado-modelo-datos-tipado-iteradores scientific
REV-0003: cap03-python-experto-asyncio-concurrencia-metaclases-packaging-proyectos scientific
REV-0004: cap04-javascript-fundamentos-closures-prototipos-nivel-intermedio scientific
REV-0005: cap05-javascript-avanzado-event-loop-promesas-web-node-proyectos scientific
REV-0006: cap06-typescript-fundamentos-sistema-tipos-estructural-uniones scientific
REV-0007: cap07-typescript-avanzado-experto-mapped-conditional-infer-proyectos scientific
REV-0008: cap08-interoperabilidad-comparativa-arquitectura-python-js-ts scientific

Apéndice E

Plan de estudio

Monografía Autónoma de Python (Python 3.12 – 3.14)

✓Monografía de Python (I – Bloques A, B y C): Historia, CPython, Entorno (uv/venv) y Nivel Principiante

- Comprender la historia de Python (Guido van Rossum, 1991), la transición Python 2 a 3 y el modelo de ejecución de bytecode de CPython (incluyendo el compilador especializador PEP 659 y el modo free-threaded PEP 703).
- Configurar entornos reproducibles conforme a PEP 517/518/621 (‘pyproject.toml‘, ‘venv‘, ‘uv‘, ‘ruff‘, ‘pyright‘, ‘pytest‘).
- Dominar la semántica de nombres vinculados a objetos (‘PyObject‘), mutabilidad vs inmutabilidad, tipos básicos y estructuras de datos nativas (‘list‘, ‘tuple‘, ‘dict‘, ‘set‘).

2 conceptos planificados.

✓Monografía de Python (II – Bloques D y E): Modelo de Datos (___dunder___), Typing Estático, Decoradores, Generadores y Context Managers

- Dominar el Data Model de Python (‘___repr___‘, ‘___eq___‘, ‘___hash___‘, ‘___len___‘, ‘___getitem___‘, ‘___enter___‘/‘___exit___‘, ‘___slots___‘).
- Diseñar jerarquías limpias con ‘@dataclass(frozen=True, slots=True)‘, protocolos estructurales (‘typing.Protocol‘) y genéricos de Python 3.12 (PEP 695).
- Construir clausuras, decoradores con ‘functools.wraps‘, generadores perezosos (‘yield‘, ‘yield from‘) y gestores de contexto (‘contextlib‘).

2 conceptos planificados.

✓Monografía de Python (III – Bloques F, G y H): Asyncio, Concurrencia, Metaprogramación, Packaging y Proyectos

- Dominar la concurrencia asíncrona con ‘asyncio‘ (‘TaskGroup‘, ‘gather‘, corrutinas nativas ‘async def‘/‘await‘) y distinguir cuándo usar ‘multiprocessing‘ o ‘concurrent.futures‘.
- Aplicar metaprogramación limpia con ‘___init_subclass___‘, metaclases, el módulo ‘ast‘ y extensiones nativas C/Rust (‘PyO3‘/‘maturin‘, ‘ctypes‘).
- Empaquetar y publicar distribuciones Wheel/sdist y estudiar el proyecto completo verificado ‘projects/python-bytecode-vm‘.

2 conceptos planificados.

Monografías Autónomas de JavaScript (ES2024+) y TypeScript (5.x)

✓Monografía de JavaScript (I – Bloques A–D): Historia, Motor V8, Ámbitos, Clausuras, Prototipos y Módulos ESM

- Comprender la evolución de ECMAScript (ES3, ES5, ES6/ES2015 hasta ES2024+) y la arquitectura JIT multipaso (Ignition + Sparkplug + Maglev + TurboFan en V8).

- Dominar los tipos primitivos (‘number‘ IEEE 754, ‘bigint‘, ‘string‘, ‘symbol‘), la igualdad estricta (‘===‘), ámbitos léxicos (‘let‘/‘const‘ y Temporal Dead Zone) y clausuras.
- Explicar con rigor la delegación prototípica (‘[[Prototype]]‘, ‘Object.create‘, sintaxis ‘class‘), la vinculación dinámica de ‘this‘ y el sistema de módulos estándar ESM (‘import‘/‘export‘).

2 conceptos planificados.

✓Monografía de JavaScript (II – Bloques E–H): Event Loop, Microtareas, Promises, Async/Await, Streams, Node.js y Proyectos

- Trazar con exactitud el orden de ejecución entre la pila síncrona, la cola de microtareas (Promises, queueMicrotask) y la cola de macrotareas (setTimeout, I/O).
- Dominar los combinadores concurrentes de Promesas (‘Promise.all‘, ‘Promise.allSettled‘, ‘Promise.race‘, ‘Promise.any‘), ‘AbortController‘, generadores asíncronos (‘for await...of‘) y Web/Node Streams.
- Gestionar fugas de memoria en el recolector generacional (Orinoco), usar ‘WeakMap‘/‘WeakRef‘, ‘Worker Threads‘ y desarrollar los 4 proyectos en JavaScript.

2 conceptos planificados.

✓Monografía de TypeScript (I – Bloques A–D): Relación con JS, Borrado de Tipos, Tipado Estructural y Uniones Discriminadas

- Comprender con exactitud la relación entre TypeScript y JavaScript: superconjunto sintáctico tipado con borrado completo de tipos (type erasure) en tiempo de ejecución.
- Configurar ‘tsconfig.json‘ con máxima rigurosidad (‘strict: true‘, ‘noUncheckedIndexedAccess: true‘, ‘exactOptionalPropertyTypes: true‘, ‘verbatimModuleSyntax: true‘).
- Dominar el subtipado estructural, tipos unión e intersección (‘|‘, ‘&‘), estrechamiento de flujo de control (*control-flow narrowing*), ‘unknown‘ vs ‘any‘ vs ‘never‘ y uniones discriminadas.

2 conceptos planificados.

✓Monografía de TypeScript (II – Bloques E–H): Programación a Nivel de Tipos (Mapped, Conditional, infer, Template Literals) y Proyectos

- Dominar los operadores avanzados del sistema de tipos de TypeScript: ‘keyof‘, ‘typeof‘, tipos indexados ‘T[K]‘, predicados de tipo (‘x is T‘), funciones de aserción (‘asserts x is T‘) y el operador ‘satisfies‘.
- Construir transformaciones a nivel de tipos mediante Mapped Types (‘{ [K in keyof T]: ... }‘), Conditional Types (‘T extends U ? X : Y‘), inferencia estructural (‘infer R‘) y Template Literal Types.
- Integrar validación de fronteras en tiempo de ejecución (Zod / TypeBox) y auditar el proyecto completo ‘projects/ts-reactive-http‘.

2 conceptos planificados.

✓Arquitectura Comparada, Interoperabilidad (FFI, WebAssembly, REST/JSON) y Rúbrica de Python, JavaScript y TypeScript

- Comparar rigurosamente los modelos de ejecución, sistemas de tipos, concurrencia y ecosistemas de Python, JavaScript y TypeScript.
- Diseñar contratos de interoperabilidad entre backends en Python (FastAPI/Pydantic) y clientes/servidores en TypeScript mediante esquemas OpenAPI 3.1 y JSON Schema.
- Superar la evaluación integradora del Volumen V resolviendo problemas de rendimiento, tipado y concurrencia en los tres lenguajes.

2 conceptos planificados.

Bibliografía

[1] Alan M. Turing – On Computable Numbers, with an Application to the Entscheidungsproblem (1936). https://www.cs.virginia.edu/~robins/Turing_Paper_1936.pdf

[2] John von Neumann – First Draft of a Report on the EDVAC (1945). https://web.mit.edu/STS.035/www/PDFs/edvac.pdf

[3] IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019) / Wikipedia Summary. https://en.wikipedia.org/wiki/IEEE_754

[4] RFC 3629 – UTF-8, a transformation format of ISO 10646 (IETF Standard). https://datatracker.ietf.org/doc/html/rfc3629

[5] ACM / IEEE-CS / AAAI Computer Science Curricula 2023 (CS2023). https://csed.acm.org/

[6] MIT OpenCourseWare 6.006 – Introduction to Algorithms. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/

[7] The Rust Programming Language (Klabnik & Nichols – Official Rust Documentation). https://doc.rust-lang.org/book/

[8] The Rust Reference – Memory Model, Type System, Lifetimes, and Unsafe Rust. https://doc.rust-lang.org/reference/

[9] The Rustonomicon – The Dark Arts of Unsafe Rust. https://doc.rust-lang.org/nomicon/

[10] C Programming Language Reference (ISO/IEC 9899 C17/C23 – cppreference). https://en.cppreference.com/w/c

[11] C++ Reference (ISO/IEC 14882 C++20/C++23 – cppreference). https://en.cppreference.com/w/cpp

[12] The Python Language Reference & Standard Library Documentation (Python 3.12–3.14). https://docs.python.org/3/reference/index.html

[13] ECMA-262 – ECMAScript 2024 Language Specification. https://tc39.es/ecma262/

[14] The TypeScript Handbook – Structural Typing, Narrowing, and Type Manipulation. https://www.typescriptlang.org/docs/handbook/intro.html

[15] Microsoft .NET & C# Language Specification (ECMA-334 / .NET 8 & 9). https://learn.microsoft.com/en-us/dotnet/csharp/

[16] The Java Language Specification & JVM Specification (Java SE 21 LTS). https://docs.oracle.com/javase/specs/jls/se21/html/index.html

[17] The Go Programming Language Specification & Memory Model. https://go.dev/ref/spec

[18] Kotlin Language Documentation & Coroutines Guide (Kotlin 2.0). https://kotlinlang.org/docs/home.html

[19] The Swift Programming Language (Swift 6.0 Official Book). https://docs.swift.org/swift-book/documentation/the-swift-programming-language/

[20] E. F. Codd – A Relational Model of Data for Large Shared Data Banks (1970) & PostgreSQL Docs. https://www.postgresql.org/docs/current/

[21] SQLite 3.54 Transactional SQL Database Engine Documentation. https://www.sqlite.org/docs.html

[22] GNU Bash Reference Manual & IEEE Std 1003.1 POSIX Shell Specification. https://www.gnu.org/software/bash/manual/bash.html

[23] WHATWG HTML Living Standard & W3C CSS Specifications. https://html.spec.whatwg.org/multipage/

[24] RFC 8259 – The JavaScript Object Notation (JSON) Data Interchange Format & YAML 1.2.2. https://datatracker.ietf.org/doc/html/rfc8259

[25] IEEE Computer Society – Guide to the Software Engineering Body of Knowledge (SWEBOK v4.0). https://www.computer.org/education/bodies-of-knowledge/software-engineering

[26] Referencia Normativa src-python-ref. https://doc.rust-lang.org/reference/src-python-ref.html

[27] Referencia Normativa src-unicode-15. https://doc.rust-lang.org/reference/src-unicode-15.html

[28] Referencia Normativa src-gamma-gof. https://doc.rust-lang.org/reference/src-gamma-gof.html

[29] Referencia Normativa src-kleppmann-ddia. https://doc.rust-lang.org/reference/src-kleppmann-ddia.html

[30] Referencia Normativa src-ecmascript-2024. https://doc.rust-lang.org/reference/src-ecmascript-2024.html

[31] Referencia Normativa src-mdn-html-css. https://doc.rust-lang.org/reference/src-mdn-html-css.html

[32] Referencia Normativa src-pierce-tapl. https://doc.rust-lang.org/reference/src-pierce-tapl.html